

Esame OS Internals (Cabodi)

Data: 25/06/2024

1 SECTION 1: THEORETICAL (Silberschatz)

Domanda 1 (Politiche di Sostituzione e Working Set)

Q1-Q3) Working Set:

1. La politica di sostituzione pagine detta "working set" usa un resident set a dimensione fissa (TRUE/FALSE). Fornire una breve motivazione.
2. Perché è difficile implementarla (la politica working set)? (scegliere la/e risposta/e giusta/e, sono possibili più risposte):
 - a. Perché la tecnica richiede di tenere, per ogni pagina, una lista dei tempi di accesso
 - b. Perché il resident set deve essere aggiornato anche in caso di assenza di PF
 - c. Perché la tecnica deve tenere traccia del tempo di ultimo accesso per ogni pagina
 - d. Perché è difficile indovinare un tempo delta adeguato

Q4) PT vs IPT (Calcoli dimensionali):

Consider a process having a virtual address space of 48 GB, in a system equipped with 16GB of RAM, with 64-bit architecture (Byte addressing) and paged memory management (4KB pages/frames) for user processes only, so not for the kernel. Assume 4GB of RAM is statically allocated to the kernel. You want to compare a standard page table-based solution (one table for each process) and an IPT-based solution. Let's assume that the pid of a process is represented on 10 bits, and that 4 extra bits (e.g valid, dirty, etc.) are needed for each entry (for both PT and IPT). Let's call x the number of bits needed for page and/or frame indexes (for both PT and IPT). Calculate:

- A) The lower bound (minimum possible value) for x , and the size of the PT (single level) for the process and of the IPT (use for x the lower bound value). Take into account that, due to alignment, an IPT or PT entry can only be 32 or 64 bits wide.
- A) Finally, using the proposed IPT, say what is the maximum possible size for the virtual address space of a process.

Q5) Algoritmo LRU a Stack:

Si consideri una strategia LRU basata su un'implementazione a stack (con 5 frame). Data la seguente stringa di riferimento: 4, 6, 4, 1, 7, 8, 2, 2, 3 (T1), 4, 2 (T2).

Mostrare lo stack al tempo T1 (dopo aver effettuato l'accesso alla pagina 3) e T2 (dopo aver effettuato l'accesso alla pagina 2). Rappresentare lo stack con numeri separati da virgole, con lo stack top per primo (ad es. 1,2,3,4,5 significa che 1 è in cima allo stack).

Q6-Q8) Page Faults Probabilities:

Si considerino due stringhe di riferimento $w1$ e $w2$ di uguale lunghezza. Sappiamo che $p2 = 2 \cdot p1$

(probabilità delle stringhe). Vengono valutati due algoritmi di sostituzione delle pagine: A1 e A2. A1 produce lo stesso numero di page fault ($F1 = F2 = 5000$) con $w1$ e $w2$. Nel caso di A2, $F1$ e $F2$ cambiano, ma la loro somma rimane invariata. Poiché la frequenza PF con A1 è peggiore del 20% rispetto ad A2, alla fine viene selezionato A2. Calcolare i valori di $F1$ e $F2$ misurati con l'algoritmo A2. Mostrare i passaggi intermedi.

Domanda 2 (File System, Blocchi e Frammentazione)

Q9) Esercizio File Binario:

Dato un file binario di dimensioni 23033 KB, calcolare esattamente il numero di blocchi di indice e blocchi di dati occupati dal file. Calcolare anche la frammentazione interna, sia per i blocchi di dati che per i blocchi di indice.

Q10-Q11) Esercizio Record a Lunghezza Variabile:

Un file di testo B, di dimensione 15300 B, contiene una sequenza di righe di lunghezza variabile, ciascuna terminata con `'\n'`. Sappiamo che la lunghezza media della riga è di 50 caratteri (`'\n'` escluso) e la lunghezza massima della riga è 100. Calcolare il numero di righe nel file (nel caso in cui il numero sia compreso tra un valore minimo e un valore massimo, calcolare l'intervallo). Consideriamo il file B appena calcolato:

- Il formato di record a lunghezza variabile influisce sull'allocazione?
- Tutti i blocchi allocati memorizzano la stessa quantità di righe di file, data dalla dimensione di un blocco divisa per la lunghezza massima della riga?

Q12) Esercizio Layout Disco e Bitmap:

A given disk is organized with physical and logical blocks of size 8KB. The disk contains multiple partitions: partition A, of NB blocks, is formatted for a file system that statically allocates NM blocks for metadata (that include directories, file control blocks and a bitmap, for free space management), and ND blocks for file data. The bitmap has one bit for each of the ND data blocks. $NM/4$ of the metadata blocks are reserved to the bitmap. Answer the following questions:

- A) Compute the ratio ND/NM .
 - A) Suppose the bitmap indicates a 33.33% ratio free/used blocks (so 1 free block for 3 used), compute (as a function of NM) the maximum size for a contiguous interval of free blocks, assuming the most favorable bitmap configuration (favorable means able to obtain larger free partitions). Give the same answer when assuming the less favorable bitmap configuration.
-

Domanda 3 (Paginazione, TLB e Slab Allocator)

Q13-Q14) Gestione Memoria (Rispondere SI/NO con breve motivazione):

Prendere in considerazione la gestione della memoria con paginazione e una MMU con un TLB. Per ciascuna delle seguenti domande, rispondere SI o NO e motivare:

- La frammentazione aumenta quando aumentano le dimensioni della pagina, perché sono necessarie partizioni contigue più grandi?
- Una free list di pagine ha una frammentazione interna media di mezza pagina?
- Tutte le strutture dati del kernel richiedono un'allocazione contigua?
- L'allocatore "slab" utilizza solo dimensioni potenza di 2?

- Il prepaging è vantaggioso solo se la probabilità che una pagina prepaginata venga utilizzata in modo efficace è $>$ dell'80%?
- La copertura della TLB (TLB reach) diminuisce all'aumentare delle dimensioni della pagina?

2 SECTION 2: OS161

Domanda 4 (Indirizzi, Configurazione e dumbvm)

Q15) Classificazione Indirizzi:

Per ciascuno dei seguenti indirizzi, dire se possono essere un indirizzo utente logico, un indirizzo logico del kernel, o un indirizzo fisico:

- 0x80803005 (User Logico / Kernel Logico / Fisico)
- 0x312010 (User Logico / Kernel Logico / Fisico)
- 0x532100 (User Logico / Kernel Logico / Fisico)

Q16) Configurazione `conf.kern`:

An OS161 system is given. Suppose you have added the following instructions to `kern/conf/conf.kern`:
`defoption project`

`optfile project syscall/project.c`

and you have created the `PROJECT` file in `kern/conf`, copied from the `DUMBVM` file.

- B) Tell if the actions described above, plus the execution (in `kern/conf`) of `./config PROJECT`, are sufficient for the optional file `syscall/project.c` to be compiled when you execute (in `kern/compile/PROJECT`) the commands `bmake depend` and `bmake`.
- B) Which one between the `project.h` and `opt-project.h` files is automatically generated by the command `./config PROJECT`? Is it always generated, or only if the following instruction appears in `PROJECT`? (`options project`)
- B) What does the file contain (the automatically generated one, referred by the previous question)?

Q17-Q18) Traduzione Indirizzo OS161:

Dato l'indirizzo logico user 0x4010, convertirlo nell'indirizzo fisico correlato. È noto che `as->as_pbase1`, `as->as_pbase2`, `as->as_vbase1`, `as->as_vbase2`, `as->as_npages1`, `as->as_npages2` hanno i seguenti valori: 0x100000, 0x200000, 0x3000, 0x6000, 2, 4. La risposta va data in forma esadecimale (0x...). Fornire formule e/o passaggi intermedi.

Q19-Q20) Funzione `ram_stealmem`:

Physical memory in `dumbvm` is allocated by calling function `ram_stealmem`. For each of the following sentences, answer true/false (provide a short explanation):

- The allocation scheme used by `ram_stealmem` will not allow interleaving of kernel and user physical RAM contiguous partitions (in other words kernel partitions and user partitions will never be interleaved).
- When using a bitmap in order to keep track of allocated/free RAM, a second array storing the sizes of allocated partitions is required for allocation of both kernel dynamic memory and user processes.
- The stack of a user process is not allocated (in `dumbvm`) calling `ram_stealmem`, because the size of the stack is a constant, and the stack is always located at the same logical addresses.
- Though the solution proposed in order to support memory de-allocation (`freeppages`) is based on a bitmap, a linked list could be used as well.
- Function `ram_stealmem` incrementally allocates memory starting at the first available address after loading the kernel.
- Function `ram_stealmem` is called for memory allocation of both kernel (dynamic memory) and user processes.

Domanda 5 (Syscalls, Trapframe e Locks)

Q21) Funzione `sys_getpid` (True/False):

Consider function `sys_getpid` (prototype: `pid_t sys_getpid(void);`) implemented in lab4. For each of the following sentences, answer true/false:

- The function is needed in order to implement the `getpid` system call, and it will be directly called by a user process.
- The function is called by `sys_exit`, in order to know the pid of the process to be signalled.
- The function returns the pid of the current process.
- The function is called to obtain the pid of another process (the one to be waited for termination).
- The function is needed in order to implement `sys_waitpid`, because the pid of the waiting process is needed.
- The function is needed in order to implement the `getpid` system call, and it will be called by the kernel, in function `syscall`.

Q22/23) Architettura Processi OS161:

Consider a user process on OS161 (for all questions provide an answer and a motivation/explanation).

- B) Which one, between `trapframe` and `switchframe`, is used to handle a system call?
- B) Is the `trapframe` allocated in the user stack or the kernel-level process stack? (motivate)
- B) Why are I/O devices mapped to `kseg1`? Could an I/O device be mapped to `kseg0`? (motivate the answer)

Q24-Q26) Implementazione Lock (Scegliere opzioni corrette con motivazione):

- Il lock può essere implementato da: 1) Un semaforo binario, più un elemento/requisito aggiuntivo. 2) Da un semaforo binario, senza alcun altro elemento/requisito. 3) Da un wait channel. 4) Da una condition variable.
- L'implementazione della funzione `lock_acquire` può includere una chiamata a: 1) Le funzioni `spinlock_data_get` e `spinlock_data_testandset`. 2) La funzione `wchan_sleep`. 3) La funzione `cv_wait`. 4) La funzione `P` su un semaforo.

SOLUZIONI

Domanda 1 (Politiche di Sostituzione e Working Set)

Q1-Q3) Working Set:

1. **FALSO**. Sebbene il parametro temporale di osservazione (la finestra Δ) sia un valore fisso e statico, la dimensione del *resident set* (ovvero il numero effettivo di frame fisici allocati in RAM per il processo in un dato istante) è **dinamica**. Il modello del working set fa sì che il numero di frame cresca o si riduca continuamente in base al numero di pagine uniche effettivamente referenziate dal processo durante l'ultima finestra temporale Δ . Questo adattamento dinamico è proprio ciò che permette al sistema di prevenire il trashing.

Q2) Difficoltà di implementazione della politica Working Set:

Le risposte corrette sono **b** e **d**. Di seguito la validazione delle singole opzioni:

- **a. FALSO**: Il modello non richiede di mantenere una *lista* completa di tutti i tempi di accesso per ogni singola pagina. È sufficiente sapere se la pagina è stata referenziata o meno all'interno della finestra temporale corrente.

- **b. VERO:** Questo è il problema architetturale principale. Nelle politiche tradizionali (demand paging), l'OS interviene per modificare le allocazioni solo al verificarsi di un Page Fault. Nel modello Working Set puro, l'OS dovrebbe monitorare continuamente lo scorrere del tempo per rimuovere le pagine non appena "scadono" (escono dalla finestra Δ), richiedendo un overhead continuo e insostenibile anche in totale assenza di page fault.
- **c. FALSO:** Non è strettamente necessario registrare il *tempo esatto* dell'ultimo accesso per ogni pagina. L'implementazione può essere approssimata in modo molto più efficiente utilizzando i bit di riferimento hardware (reference bits) campionati a intervalli regolari tramite interrupt del timer.
- **d. VERO:** La scelta del parametro Δ è un problema di tuning complesso. Un Δ troppo piccolo non copre l'intera località del processo (causando thrashing), mentre un Δ troppo grande include pagine di località precedenti ormai inutili (spreca RAM preziosa). Essendo il comportamento dei programmi dinamico, un Δ fisso è raramente ottimale per l'intera esecuzione.

Q4) PT vs IPT (Calcoli dimensionali):

- **A) Lower Bound e Dimensioni:** La RAM disponibile per i processi utente è 16GB – 4GB (kernel) = 12GB.
 - *Frame fisici (IPT):* $12\text{GB}/4\text{KB} = 3 \times 2^{20} = 3\text{M}$ frame. Il *lower bound* per l'indice di frame è $z = \lceil \log_2(3\text{M}) \rceil = 22$ bit.
 - *Pagine virtuali (PT):* $48\text{GB}/4\text{KB} = 12 \times 2^{20} = 12\text{M}$ pagine. Il *lower bound* per l'indice di pagina è $z = \lceil \log_2(12\text{M}) \rceil = 24$ bit.
 - *Dimensione PT:* Entry = 22 (PFN) + 4 (extra) = 26 bit $\rightarrow$ 32 bit (4 Byte). Dimensione totale = $12\text{M} \times 4\text{B} = 48 \text{ MB}$.
 - *Dimensione IPT:* Entry = 10 (PID) + 24 (VPN) + 4 (extra) = 38 bit $\rightarrow$ 64 bit (8 Byte). Dimensione totale = $3\text{M} \times 8\text{B} = 24 \text{ MB}$.
- **B) Spazio Virtuale Massimo con IPT:** Avendo vincolato l'entry della IPT a 64 bit, e sottraendo 10 bit per il PID e 4 bit extra, rimangono 50 bit disponibili per l'indice di pagina (VPN). Il numero massimo di pagine indirizzabili è quindi 2^{50} . Lo spazio virtuale massimo è $2^{50} \times 4\text{KB} = 2^{62} \text{ Byte} = 4 \text{ Exabyte (EB)}$.

Q5) Algoritmo LRU a Stack: La politica LRU sposta la pagina referenziata in cima allo stack. Rispettando la convenzione del testo (stack top per primo):

- **Stack a T1:** 3, 2, 8, 7, 1
- **Stack a T2:** 2, 4, 3, 8, 7

Q6-Q8) Page Fault Probabilities:

Siano le probabilità $p_1 = 1/3$ e $p_2 = 2/3$ (poiché $p_2 = 2 \cdot p_1$ e $p_1 + p_2 = 1$).

Il Valore Atteso dei PF per A_1 è $E(A_1) = 5000$. Poiché A_1 è peggiore del 20% rispetto ad A_2 , abbiamo $5000 = 1.2 \cdot E(A_2)$, da cui $E(A_2) = 4166.67$.

Impostiamo il sistema per l'algoritmo A_2 :
$$\begin{cases} F_1 + F_2 = 10000 \\ \frac{1}{3}F_1 + \frac{2}{3}F_2 = 4166.67 \end{cases} \implies F_1 + 2F_2 = 12500$$

Sostituendo $F_2 = 10000 - F_1$ nella seconda equazione:

$$F_1 + 2(10000 - F_1) = 12500 \implies 20000 - F_1 = 12500 \implies$$

Domanda 2 (File System, Blocchi e Frammentazione)

Q9) Esercizio File Binario:

- **Calcolo Blocchi Dati e Frammentazione Dati:**

Dimensione file = 23033 KB. Dimensione blocco = 4 KB.

Blocchi Dati = $\lceil 23033/4 \rceil = \mathbf{5759 \text{ blocchi dati}}$.

Spazio fisico allocato per i dati = $5759 \times 4 \text{ KB} = 23036 \text{ KB}$.

Frammentazione interna dati = $23036 \text{ KB} - 23033 \text{ KB} = \mathbf{3 \text{ KB}}$.

- **Calcolo Blocchi Indice:**

Ogni blocco indice da 4 KB può contenere 1024 puntatori (assumendo puntatori da 4 byte standard per architetture a 32 bit). Distribuiamo i 5759 blocchi dati:

- **Direct:** 12 blocchi. (Restano $5759 - 12 = 5747$ blocchi).
- **Single Indirect:** Utilizza **1 blocco indice** per mappare 1024 blocchi. (Restano $5747 - 1024 = 4723$ blocchi).
- **Double Indirect:** Deve mappare i restanti 4723 blocchi. Occorre **1 blocco indice radice**. Questo punterà a blocchi indice di primo livello. Numero di blocchi di primo livello necessari = $\lceil 4723/1024 \rceil = \mathbf{5 \text{ blocchi indice intermedi}}$.

Totale Blocchi Indice allocati = 1 (Single)+1 (Root Double)+5 (Intermedi) = **7 blocchi indice**.

- **Calcolo Frammentazione Interna Blocchi Indice:**

La frammentazione interna degli indici è data dallo spazio (in byte) non utilizzato all'interno dei blocchi allocati parzialmente.

- Il blocco radice del Double Indirect usa 5 puntatori su 1024. Spreco: $(1024 - 5) \times 4 \text{ Byte} = 4076 \text{ Byte}$.
- Dei 5 blocchi intermedi, 4 sono pieni. Il 5° mappa i restanti dati ($4723 - 4096 = 627$ blocchi). Usa 627 puntatori su 1024. Spreco: $(1024 - 627) \times 4 \text{ Byte} = 1588 \text{ Byte}$.

Frammentazione interna indici totale = $4076 + 1588 = \mathbf{5664 \text{ Byte}}$.

Domanda 2 (File System - Parte 2)

Q10-Q11) Esercizio Record a Lunghezza Variabile:

Dati: File B = 15300 Byte. Lunghezza media = 50 caratteri (\n escluso).

- **Calcolo del numero di righe:**

Poiché la lunghezza media della riga (escluso il terminatore) è di 50 byte, la dimensione logica media di ogni record (incluso il \n) è di 51 byte. Avendo a disposizione la media esatta, il numero di righe si ottiene puntualmente:

$$\text{Numero di righe} = 15300 \text{ B} / 51 \text{ B/riga} = \mathbf{300 \text{ righe}}.$$

(La richiesta di un intervallo min/max è un distrattore: il calcolo esatto è possibile).

- **Influenza del formato variabile sull'allocazione:**

NO. Il File System alloca i blocchi fisici basandosi esclusivamente sulla dimensione totale in byte del file (15300 B). Il fatto che i record al suo interno siano a lunghezza fissa o variabile non altera il numero di blocchi (es. da 4KB) necessari a contenere i 15300 byte complessivi.

- **I blocchi memorizzano la stessa quantità di righe?**

NO. A causa della lunghezza variabile (fino a 100 caratteri), un blocco fisico riempito con righe molto lunghe conterrà un numero totale di record significativamente inferiore rispetto a un blocco riempito casualmente con righe molto corte.

Q12) Esercizio Layout Disco e Bitmap:

- **A) Calcolo del rapporto ND/NM:**

Sappiamo che la bitmap occupa $NM/4$ blocchi. Convertiamo la dimensione di un blocco in bit, poiché ogni bit mappa un blocco dati (ND):

$$1 \text{ blocco} = 8 \text{ KB} = 8192 \text{ Byte} = 65536 \text{ bit.}$$

Il numero totale di bit nella bitmap è $(NM/4) \times 65536 = 16384 \times NM$.

Poiché Bit totali = ND , otteniamo $ND = 16384 \times NM$. Pertanto, **il rapporto ND/NM è 16384 (16K).**

- **B) Dimensione massima di un intervallo contiguo di blocchi liberi:**

Il rapporto 1 blocco libero ogni 3 occupati indica che $\frac{1}{4}$ dei blocchi totali è libero.

$$ND_{\text{free}} = \frac{ND}{4} = \frac{16384 \times NM}{4} = 4096 \times NM \text{ blocchi liberi.}$$

- **Configurazione più favorevole:** Tutti i blocchi liberi sono adiacenti. La dimensione massima contigua è pari al numero totale di blocchi liberi.

$$\text{Size Max} = 4096 \times NM \text{ blocchi} \times 8 \text{ KB/blocco} = 32768 \times NM \text{ KB} = \mathbf{32 \text{ MB} \times NM}.$$

- **Configurazione meno favorevole:** Massima frammentazione. I blocchi liberi sono perfettamente intervallati da gruppi di blocchi occupati (es. U, U, U, F, U, U, U, F). La dimensione massima contigua è di **1 solo blocco (8 KB)**.

Domanda 3 (Paginazione, TLB e Slab Allocator)

Q13-Q14) Gestione Memoria (Rispondere SI/NO):

- **La frammentazione aumenta quando aumentano le dimensioni della pagina, perché sono necessarie partizioni contigue più grandi?**

NO. La paginazione, per definizione, elimina completamente la necessità di allocare memoria fisica contigua; pertanto la frammentazione esterna è nulla. Aumentando la dimensione della pagina si aumenta la frammentazione *interna* (nell'ultima pagina allocata), ma non per motivi legati alla contiguità.

- **Una free list di pagine ha una frammentazione interna media di mezza pagina?**

NO. La regola della "mezza pagina" si applica all'ultima pagina allocata a un processo. Una *free list* è una struttura che gestisce frame fisici interi, vuoti e non allocati. Non essendovi dati allocati al loro interno, il concetto di frammentazione interna è inesistente.

- **Tutte le strutture dati del kernel richiedono un'allocazione contigua?**

NO. Sebbene alcuni buffer (es. per il DMA) richiedano frame fisici contigui, molte strutture dati dinamiche del kernel non lo richiedono. Nei sistemi moderni (es. tramite `vmalloc` in Linux), il kernel alloca pagine fisiche sparse e le mappa in indirizzi logici contigui nello spazio virtuale del kernel.

- **L'allocatore "slab" utilizza solo dimensioni potenza di 2?**
NO. È il *Buddy Allocator* a frammentare la memoria in blocchi di dimensioni potenza di 2. Lo *Slab Allocator* è progettato esattamente per superare questo limite: crea cache pre-allocate basate sulla dimensione esatta (in byte) delle specifiche strutture dati del kernel (es. `struct proc`), minimizzando la frammentazione interna.
- **Il prepaging è vantaggioso solo se la probabilità che una pagina prepaginata venga utilizzata in modo efficace è > dell'80%?**
NO. La percentuale dell'80% è un valore arbitrario. Il vantaggio del prepaging dipende dal bilanciamento tra la latenza del disco e la banda passante. Poiché una singola operazione di I/O contigua (prepaging) è enormemente più economica in termini di latenza meccanica/elettronica rispetto a gestire innumerevoli page fault singoli (che richiedono un seek per ogni fault), la politica può risultare vantaggiosa anche con percentuali di utilizzo molto più basse.
- **La copertura della TLB (TLB reach) diminuisce all'aumentare delle dimensioni della pagina?**
NO. La copertura della TLB (TLB Reach) si calcola come: Numero di Entry $\times$ Dimensione Pagina. Aumentando la dimensione della pagina, la TLB copre una porzione di memoria maggiore a parità di entry hardware. Pertanto, la copertura *aumenta*.

Domanda 4 (Indirizzi, Configurazione e dumbvm)

Q15) Classificazione Indirizzi:

In architettura MIPS (OS161), lo spazio di indirizzamento è strettamente segmentato. La discriminante principale è il limite `0x80000000`: gli indirizzi virtuali inferiori a questa soglia appartengono allo spazio utente (`kuseg`), mentre quelli superiori o uguali appartengono al kernel.

- **0x80803005: Può essere SOLO un indirizzo logico del Kernel.**
Motivazione: Essendo maggiore di `0x80000000`, rientra nel segmento `kseg0`. Non può essere generato in User Mode (causerebbe un'eccezione) e non può essere fisico poiché supera di gran lunga i limiti hardware della RAM del simulatore.
- **0x312010: Può essere SIA un indirizzo User Logico SIA un indirizzo Fisico.**
Motivazione: Essendo minore di `0x80000000`, è un indirizzo virtuale perfettamente valido per lo spazio logico utente (`kuseg`). Contestualmente, il suo valore (~ 3.2 MB) lo rende un indirizzo fisico ammissibile all'interno di una tipica RAM installata (es. 8 MB).
- **0x532100: Può essere SIA un indirizzo User Logico SIA un indirizzo Fisico.**
Motivazione: Medesimo ragionamento del punto precedente. È un indirizzo logico utente (`kuseg`) sintatticamente valido ed è fisicamente plausibile (~ 5.4 MB) nei limiti della memoria RAM del sistema.

[colback=blue!5!white,colframe=blue!75!black,title=**Approfondimento Teorico: Classificazione Indirizzi in OS161 (MIPS)**] In architettura MIPS, un indirizzo è formalmente solo un valore numerico esadecimale. La sua natura (Fisica o Logica) è determinata dall'intersezione tra le regole della CPU e l'hardware effettivamente installato:

1. Limite Architetturale (CPU MIPS a 32 bit):

- **Da 0x00000000 a 0x7FFFFFFF (Primi 2 GB):** Spazio `kuseg`. Tutti gli indirizzi generati in User Mode devono cadere in questo intervallo.
- **Da 0x80000000 a 0xFFFFFFFF (Ultimi 2 GB):** Segmenti del **Kernel** (`kseg0`, `kseg1`, ecc.). Inaccessibili dallo spazio utente.

2. Limite Hardware (Simulatore sys161):

- La memoria RAM fisica installata è tipicamente di **8 MB** (8×2^{20} byte).
- Gli indirizzi fisici reali esistono unicamente nell'intervallo che va da `0x00000000` a `0x007FFFFFFF`. Qualsiasi indirizzo fisico superiore a questo valore punta a una porzione di hardware inesistente.

Regola d'Oro per la Classificazione:

Combinando i due limiti, si ottiene lo schema di discriminazione assoluto:

- `< 0x007FFFFFFF (0 - 8 MB)`: Rientra nella RAM installata ed è sotto la soglia kernel. Può essere **SIA** Indirizzo Logico Utente **SIA** Indirizzo Fisico.
- `> 0x007FFFFFFF e < 0x80000000 (8 MB - 2 GB)`: Sfora la RAM fisica ma rispetta il limite utente. È **SOLO** Indirizzo Logico Utente.
- `≥ 0x80000000 (2GB-4GB)` : *Supera i limiti dei privilegi utente. SOLO Indirizzo Logico Kernel.*

Domanda 4 (Indirizzi, Configurazione e dumbvm)

Q16) Configurazione `conf.kern`:

- **A) Le azioni descritte sono sufficienti per la compilazione di `project.c`?**
FALSO. Affinché il file `syscall/project.c` venga effettivamente incluso nel processo di compilazione (`bmake`), non basta definire l'opzione (`defoption`) e mappare il file (`optfile`) in `conf.kern`. È strettamente necessario inserire la direttiva `options project` all'interno del file di configurazione specifico (PROJECT) *prima* di eseguire lo script `./config PROJECT`. Senza di essa, lo script non inserirà il file nel Makefile generato.
- **B) Quale file viene generato, e viene generato sempre?**
Il file generato automaticamente è `opt-project.h`. Questo file viene **generato sempre** durante l'esecuzione di `./config PROJECT`, a prescindere dalla presenza della direttiva `options project` nel file di configurazione (è sufficiente che esista la `defoption` in `conf.kern`).
- **C) Cosa contiene il file generato?**
Il file contiene una macro C generata in base allo stato dell'opzione. Nello specifico:
 - Conterrà `#define OPT_PROJECT 1` se la riga `options project` è presente nel file PROJECT.
 - Conterrà `#define OPT_PROJECT 0` se l'opzione non è stata attivata.

Q17-Q18) Traduzione Indirizzo OS161:

Dati preliminari (Dimensione Pagina = 4 KB = 0x1000 byte):

- **Segmento 1:** `vbase1 = 0x3000`, `npages1 = 2` (0x2000 byte). Range logico valido: da `0x3000` a `0x4FFF`. `pbase1 = 0x100000`.
- **Segmento 2:** `vbase2 = 0x6000`, `npages2 = 4` (0x4000 byte). Range logico valido: da `0x6000` a `0x9FFF`. `pbase2 = 0x200000`.

Calcolo per l'indirizzo logico user 0x4010:

1. **Identificazione del Segmento:** Controllando i limiti, si nota che $0x3000 \leq 0x4010 \leq 0x4FFF$. L'indirizzo rientra pertanto nel **Segmento 1**.

2. **Calcolo dell'Offset:** L'offset interno al segmento si ottiene sottraendo la base virtuale dall'indirizzo target:

$$\text{Offset} = 0x4010 - 0x3000 = 0x1010$$

3. **Calcolo dell'Indirizzo Fisico:** Si somma l'offset calcolato alla base fisica del segmento corrispondente:

$$\text{Indirizzo Fisico} = \text{pbase1} + \text{Offset} = 0x100000 + 0x1010 = 0x101010$$

Risultato Finale: 0x101010

Q19-Q20) Funzione ram_stealmem:

- **Lo schema di allocazione non permette l'interleaving di partizioni contigue kernel e utente?**

FALSO. `ram_stealmem` alloca la memoria in modo strettamente sequenziale (bump allocator) man mano che arrivano le richieste. Poiché a runtime richieste del kernel (tramite `kmalloc`) e richieste utente (creazione di nuovi address space) si alternano dinamicamente, le loro partizioni fisiche in RAM risulteranno inevitabilmente interlacciate (interleaved).

- **Usando una bitmap, è necessario un secondo array per salvare le dimensioni delle partizioni allocate?**

VERO. Le funzioni di deallocazione (come `freeppages(addr)`) ricevono solo l'indirizzo base e non il numero di pagine da liberare. Poiché una semplice bitmap non marca i "confini" di un'allocazione contigua multipla, un array parallelo è indispensabile per indicare al sistema quanti bit azzerare a partire dall'indirizzo dato.

- **Lo stack utente non è allocato chiamando ram_stealmem perché l'indirizzo logico è costante?**

FALSO. Anche se l'indirizzo *logico* (virtuale) di partenza dello stack è una costante (es. `USERSTACK`), il sistema deve allocare frame *fisici* reali e distinti per lo stack di ogni nuovo processo, altrimenti i processi sovrascriverebbero l'uno i dati dell'altro. Questa allocazione fisica avviene chiamando `getppages`, che in `dumbvm` utilizza proprio `ram_stealmem`.

- **Al posto della bitmap si potrebbe usare una linked list per supportare la deallocazione?**

VERO. Una "Free List" (lista concatenata di blocchi di memoria liberi) è la principale struttura dati alternativa alla bitmap per la gestione della memoria libera, utilizzata tipicamente nei gestori basati su politiche come First-Fit, Best-Fit o Buddy System.

- **ram_stealmem alloca incrementalmente a partire dal primo indirizzo disponibile dopo il caricamento del kernel?**

VERO. La funzione opera aggiornando una variabile puntatore (generalmente inizializzata a `firstaddr` dal codice di bootstrap dipendente dalla macchina), la quale si posiziona esattamente subito dopo il termine della sezione BSS dell'immagine del kernel in memoria.

- **ram_stealmem è chiamata per allocare memoria sia per il kernel sia per i processi utente?**

VERO. In `dumbvm`, questa funzione è l'unico motore per recuperare frame fisici. Viene chiamata indirettamente tramite `getppages` sia dalla `kmalloc` (strutture dinamiche kernel), sia dalle funzioni `as_prepare_load` e `as_define_stack` (creazione di un processo utente).

Domanda 5 - Parte 1

Q21) Funzione `sys_getpid` (True/False):

- **FALSO.** (*The function is needed... and it will be directly called by a user process*). Un processo in User Mode non può invocare direttamente funzioni del Kernel. L'utente chiama il wrapper di libreria C, che solleva una *trap* (interrupt software) per attraversare la barriera dei privilegi.
- **FALSO.** (*The function is called by `sys_exit`...*). All'interno del kernel, `sys_exit` ha accesso diretto alla struttura del processo corrente (es. tramite `curproc`) per impostare i segnali o lo stato, senza dover invocare `sys_getpid`.
- **VERO.** (*The function returns the pid of the current process*). Il suo unico scopo è leggere il PID dalla struttura `curproc` in esecuzione e restituirlo.
- **FALSO.** (*The function is called to obtain the pid of another process*). La funzione ha prototipo `void`, non accetta argomenti e restituisce esclusivamente il PID del processo chiamante.
- **FALSO.** (*The function is needed in order to implement `sys_waitpid`...*). Il kernel in `sys_waitpid` possiede già tutte le informazioni di contesto. Il PID del chiamante è noto tramite `curproc`, e il PID del processo atteso viene passato come argomento dalla system call.
- **VERO.** (*The function is needed in order to implement the `getpid`... and it will be called by the kernel...*). Questa è la descrizione esatta del flusso di esecuzione: la *trap* trasferisce il controllo alla funzione dispatcher `syscall` nel kernel, la quale decodifica il numero della system call e invoca fisicamente `sys_getpid`.

Domanda 5 - Parte 2

Q22/23) Architettura Processi OS161:

- **A) Quale struttura, tra `trapframe` e `switchframe`, gestisce una system call?**

Risposta: `Trapframe`.

Motivazione: Una system call genera un'eccezione hardware (trap) che forza la transizione da User Mode a Kernel Mode. Il `trapframe` è la struttura dati utilizzata per salvare lo stato esatto dei registri utente nel momento esatto dell'interruzione. Lo `switchframe`, invece, viene utilizzato esclusivamente per i Context Switch tra thread già in esecuzione in Kernel Mode.

- **B) Il `trapframe` è allocato nello stack utente o nello stack kernel?**

Risposta: Nello stack del Kernel.

Motivazione: È una questione di sicurezza e stabilità assoluta. Il puntatore allo stack utente (`sp`) è sotto il controllo dell'applicazione. Se un programma utente malevolo o baccato alterasse il proprio `sp` puntando ad un'area non valida o protetta, e il kernel tentasse di scrivervi il `trapframe`, si verificherebbe un crash irreversibile (Kernel Panic). Utilizzando il proprio stack privato, il kernel si garantisce uno spazio di memoria fidato e sicuro.

- **C) Perché i dispositivi di I/O sono mappati in `kseg1`? Potrebbero essere mappati in `kseg0`?**

Risposta: Devono essere in `kseg1` perché è un segmento NON cachato (uncached).

Motivazione: L'architettura MIPS utilizza il *Memory-Mapped I/O (MMIO)*, comunicando con l'hardware tramite la lettura e scrittura su specifici indirizzi fisici.

- Se l'I/O fosse mappato in `kseg0` (che è soggetto a caching), la CPU finirebbe per leggere valori vecchi dalla cache invece dello stato hardware reale e aggiornato (es. non vedrebbe la pressione di un tasto). Inoltre, le scritture di comando verso un dispositivo verrebbero ritardate o trattenute nella cache (write-back), bloccando la comunicazione con la periferica.
- Mappando in `kseg1`, si forza la CPU a bypassare le cache L1/L2, garantendo che ogni operazione di load/store arrivi fisicamente e istantaneamente ai registri del dispositivo hardware sul bus di sistema.

Domanda 5 - Parte 3

Q24-Q26) Implementazione Lock:

Analisi delle due affermazioni proposte sulle primitive di sincronizzazione in OS161.

- **Il lock può essere implementato da: (Opzioni corrette: 1 e 3)**

Motivazione Opzione 1: Un lock può essere costruito "avvolgendo" un semaforo binario, ma necessita di un elemento aggiuntivo cruciale: la **Thread Ownership** (proprietà). A differenza dei semafori (dove un thread può fare `P()` e un altro `V()`), la semantica del lock impone che solo il thread che ha acquisito il lock possa rilasciarlo. Pertanto serve un campo aggiuntivo nella struttura per memorizzare il PID/indirizzo del thread possessore.

Motivazione Opzione 3: In OS161, l'implementazione nativa e più diretta di un lock si basa sui **wait channel** (`wchan`), ovvero le code d'attesa di basso livello fornite dal kernel su cui i thread possono essere messi in stato di *sleep* in attesa che la risorsa si liberi.

- **L'implementazione della funzione `lock_acquire` può includere una chiamata a: (Opzioni corrette: 2 e 4)**

Motivazione Opzione 2: Se il lock è implementato in modo nativo tramite wait channel, nel caso in cui la risorsa sia già occupata, la `lock_acquire` deve chiamare la funzione `wchan_sleep` per addormentare il thread corrente, togliendolo dalla CPU senza sprecare cicli di clock.

Motivazione Opzione 4: Se il lock è implementato utilizzando un semaforo binario come motore interno, l'acquisizione della risorsa avviene invocando la primitiva `P()` su tale semaforo.

*(Nota tecnica: L'opzione 1 è errata perché le primitive di basso livello come `testandset` appartengono all'implementazione degli **spinlock** (attesa attiva) forniti dall'hardware, non dei **lock** ad alto livello che prevedono invece l'attesa passiva).*